

```

1 # Foundational Bindings: Imports and Ring Definitions
2 begin
3     using Oscar: ZZ, matrix_space, symmetric_group
4     using Latexify
5 end

```

```

[(1, 2), (1, 2), (3, 2), (3, 2), (1, 4), (3, 4), (2, 4)]

```

```

1 begin
2     # Global Data-Oriented Defaults (Immutable Constants)
3     # -----
4     # We model the Seven Bridges of Königsberg as a strict ADT (Vertices, Edges)
5     # Vertices: 1=North Bank, 2=Island, 3=South Bank, 4=East Bank
6     K_Vertices = [1, 2, 3, 4]
7
8     # Edges: Undirected Multigraph connections
9     K_Edges = [
10         (1, 2), (1, 2), # Two bridges from North to Island
11         (3, 2), (3, 2), # Two bridges from South to Island
12         (1, 4),           # One bridge from North to East
13         (3, 4),           # One bridge from South to East
14         (2, 4)            # One bridge from Island to East
15     ]
16 end

```

Lecture Sixteen: The Birth of Graph Theory

Scope

This lecture introduces graph theory, tracing its origins to 1736 with Leonhard Euler and the foundational Seven Bridges of Königsberg problem. We transition from the internal structure of numbers (Number Theory) to the structural relationships *between* discrete entities.

Under our **FP-DOD** methodology, a graph $G = (V, E)$ is not a mutating object of pointers, but an immutable algebraic schema evaluated strictly via stateless pipelines and matrix geometries.

I. Modeling Relationships (The Graph ADT)

A graph is composed of **vertices** (discrete entities) and **edges** (connections). Euler abstracted the landmasses of Königsberg into vertices and the bridges into edges, proving that physical distances are mathematically irrelevant; only relational topology matters.

Basic Graph Types

To model relationships mathematically, we classify graphs by their strict edge properties:

- **Undirected Graphs:** Edges have no direction (two-way relationships). Represented algebraically as unordered sets $\{u, v\}$.
- **Directed Graphs (Digraphs):** Edges point from one vertex to another. Represented as strict ordered tuples (u, v) .
- **Simple Graphs:** Graphs with no loops (edges connecting a vertex to itself) and no multiple edges.
- **Multigraphs:** Graphs that allow multiple edges between the exact same pair of vertices (The Seven Bridges of Königsberg is an undirected multigraph).

Königsberg Topology Proof:

- **Vertex Degrees:** **[3, 5, 3, 3]**

(Notice that every single vertex has an odd degree: 3, 5, 3, 3)

```
begin
  # 1. Pure FP (Stateless Property Evaluation)
  # We evaluate the degree (number of connections) of a vertex using a pure
  fold/count
  degree_fp(v::Int, edges::Vector{Tuple{Int, Int}}) =
    count(e -> e[1] == v || e[2] == v, edges)

  # Map the degree function statelessly across the entire vertex domain
  k_degrees = map(v -> degree_fp(v, K_Edges), K_Vertices)

  Markdown.parse("""
  ### Königsberg Topology Proof:
  * **Vertex Degrees:**  $\mathbf{\{k\_degrees\}}$ 
  *(Notice that every single vertex has an odd degree: 3, 5, 3, 3)*
  """)
end
```

II. Eulerian Circuits and Parity

Euler proved that traversing a network perfectly (crossing every edge exactly once) relies strictly on structural parity, not pathfinding algorithms:

- **Eulerian Circuit** (starts and ends at the same node): Exists iff **all** vertices have an **even** degree.
- **Eulerian Path** (starts and ends at different nodes): Exists iff exactly **two** vertices have an **odd** degree.

The Königsberg Solution:

- **Has Eulerian Circuit? false**
- **Has Eulerian Path? false**

Conclusion: Because all 4 vertices have odd degrees, it is mathematically impossible to walk the Seven Bridges of Königsberg.

```
begin
  # 1. Pure FP (Stateless Eulerian Filters)
  # We reduce complex graph traversals into O(V) boolean filters
  has_eulerian_circuit_fp(degrees) = all(d -> mod(d, 2) == 0, degrees)
  has_eulerian_path_fp(degrees) = count(d -> mod(d, 2) != 0, degrees) == 2

  ans_circuit = has_eulerian_circuit_fp(k_degrees)
  ans_path    = has_eulerian_path_fp(k_degrees)

  Markdown.parse("""
  ### The Königsberg Solution:
  * **Has Eulerian Circuit?**  $\mathbf{\$ans\_circuit}$ 
  * **Has Eulerian Path?**  $\mathbf{\$ans\_path}$ 

  **Conclusion:** Because all 4 vertices have odd degrees, it is mathematically
  impossible to walk the Seven Bridges of Königsberg.
  """)
end
```

III. The Handshaking Lemma & Matrix Geometry

A fundamental theorem of graph combinatorics states that the sum of all vertex degrees must exactly equal twice the number of edges: $\sum_{v \in V} \deg(v) = 2|E|$

Furthermore, we can map this topology into **Abstract Algebra** by projecting the edge list into an Adjacency Matrix over the Integer Ring $\mathbb{Z}$.

Structural Verification:

- **Handshaking Lemma Proof: true** (Sum of degrees is exactly 14)

Adjacency Matrix over $\mathbb{Z}$: [0 2 0 1; 2 0 2 1; 0 2 0 1; 1 1 1 0]

```
begin
  # 1. Pure FP (The Handshaking Lemma Verification)
  handshake_fp(degrees, edges) = reduce(+, degrees) == 2 * length(edges)

  # 2. Abstract Algebra / Linear Algebra (Functional Matrix Construction)
  # We map the 2D matrix statelessly, then inject it into Oscar's Matrix Space
  function build_adjacency_aa(vertices, edges)
    M_Space = matrix_space(ZZ, length(vertices), length(vertices))

    # Pure FP 2D map to build adjacency counts
    adj_array = map(1:length(vertices)) do i
      map(1:length(vertices)) do j
        count(e -> (e[1] == i && e[2] == j) || (e[1] == j && e[2] == i),
          edges)
      end
    end

    # Flatten into the Abstract Algebra matrix
    return M_Space(reduce(vcat, map(row -> reshape(row, 1, length(vertices)),
      adj_array)))
  end

  ans_handshake = handshake_fp(k_degrees, K_Edges)
  k_adjacency = build_adjacency_aa(K_Vertices, K_Edges)

  Markdown.parse("""
  ### Structural Verification:
  * **Handshaking Lemma Proof:**  $\mathbf{\$(ans\_handshake)\$}$  (Sum of degrees is
  exactly  $\$(reduce(+, k\_degrees))$ )

  **Adjacency Matrix over  $\mathbb{Z}$ :**
   $\$(k\_adjacency)$ 
  """)
end
```

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283	284	285	286	287	288	289	290	291	292	293	294	295	296	297	298	299	300	301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319	320	321	322	323	324	325	326	327	328	329	330	331	332	333	334	335	336	337	338	339	340	341	342	343	344	345	346	347	348	349	350	351	352	353	354	355	356	357	358	359	360	361	362	363	364	365	366	367	368	369	370	371	372	373	374	375	376	377	378	379	380	381	382	383	384	385	386	387	388	389	390	391	392	393	394	395	396	397	398	399	400	401	402	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420	421	422	423	424	425	426	427	428	429	430	431	432	433	434	435	436	437	438	439	440	441	442	443	444	445	446	447	448	449	450	451	452	453	454	455	456	457	458	459	460	461	462	463	464	465	466
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----